

Objectifs

Comprendre l'intérêt des Sessions, des Listeners et des Filtres. Savoir les appliquer

Créez de préférence un nouveau contexte `tp502`.
Un contexte générique `tp.zip` à déployer puis renommer est à disposition sur Moodle.

Les Sessions

Exercice 1 : Le compteur

Q1. On souhaite réaliser une page web qui affiche pour chacun des utilisateurs le message *"Vous avez accédé 12 fois à cette page sur les 53 accès au total"*. Réalisez une servlet `Compteur` ou une JSP qui permet d'afficher cette phrase.

- Pour le compteur local on utilisera une session
- Pour le compteur global on utilisera un attribut de la servlet

Pour vérifier le comportement correct de votre servlet, vous utiliserez deux navigateurs différents ou éventuellement une fenêtre de navigation privée.

Q2. Exécutez une requête Curl sur cette page. Visualisez notamment les données de l'entête HTTP.

Q3. Essayez via une requête Curl, d'incrémenter le compteur local.

Les Filtres

Les représentants de l'interface `Filter` (affinée en `HttpFilter`) sont des objets qui "filtrent" les requêtes et effectuent des traitement avant l'exécution de la page appelée. Un filtre possède la forme suivante :

```
@WebFilter("/*")
public class MonFiltre extends HttpFilter {
    public void doFilter(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws IOException, ServletException
    {
        // actions à réaliser avant exec de la requête
        chain.doFilter(req, res);
        // actions à réaliser après exec de la requête
    }
}
```

Exercice 2 : MyRequestFilter

Q1. Réalisez un filtre `MyRequestFilter` qui s'applique à toutes les url du contexte et qui indique lors de chaque requête, via un message dans la console, la date et l'url utilisée pour cette requête.

Q1.1. Testez avec le compteur précédent

Q1.2. Réalisez une servlet `Test.java` qui affiche juste un `All is OK !` (à recopier du tp501 par exemple). Vérifiez le déclenchement du filtre avec les appels à cette servlet.

Q1.3. Modifiez les valeurs d'application du filtre en testant son application à

- une seule servlet
- plusieurs servlets distinctes
- toutes les servlet qui ont le même mot clé dans l'URI (genre `/servlet/maservlet`)
- Toutes les url

Exercice 3 : MyTimeFilter

Q1. Réalisez un filtre qui indique dans la console le temps passé pour chaque requête (dans chaque page). Testez ce filtre sur la servlet `test`

Q1.1. Réalisez une servlet `Lister` qui liste la table `etudiant`.

Vous devez bien sûr constater d'une part que votre filtre s'applique aussi bien aux deux servlets, sans toucher à une seule ligne de code, et d'autre part que `Lister` prend nettement plus de temps que `test` (ce qui semble normal).

Exercice 4 : MyCipherFilter

Q1. DIFFICILE!! : Un filtre de déchiffrement. Réalisez un filtre qui transforme systématiquement la valeur passée à un paramètre "password" en inversant ses lettres (si vous le souhaitez vous pouvez utiliser un chiffrement plus complexe type [AES](#), ou plus simple type "transformer en majuscules"). Les servlets ont donc la garantie de toujours avoir ce paramètre déchiffré.

Aide : Il est impossible de modifier les paramètres d'une requête : il n'y a pas de `setParameter` dans `Request`. Par contre, il est possible de surcharger la méthode `getParameter`. Pour surcharger un objet `Request` il existe des `RequestWrappers`. Ecrire comme classe interne une sous-classe de `HttpServletRequestWrapper` qui prend en paramètre la requête et qui redéfinit la méthode `getParameter()` pour inverser le password. Cette instance de requete est alors renvoyée dans le `doFilter`. C'est un exemple d'application d'un design pattern "Decorator".

Afin de tester ce filtre, créez une servlet ou une jsp qui lit un paramètre "password" et qui affiche simplement sa valeur. Passez une valeur codée (lettres à l'envers, ou minuscules) à ce paramètre dans l'url, il devrait alors s'afficher décodé (inversé, ou majuscule).

Q2. Afin de structurer votre contexte, placez tous vos filtres dans un package `filter`. Nous placerons par la suite tous les listeners dans un package `listeners`.

Les Listeners

Exercice 5 : MyRequestListener

La définition d'un `Listener` est un peu plus complexe qu'un `Filter`. Les Listeners sont décomposés en plusieurs sous-classes et chacune doit définir des méthodes spécifiques. Un oeil sur la [documentation](#) est

indispensable.

Q1. Réalisez une classe `MyRequestListener` qui affiche dans la console l'URL utilisée lors de la création d'une requête. On implémentera pour cela un `ServletRequestListener` et notamment la méthode `requestInitialized`.

1. Testez cet affichage lors des appels à vos différents servlets
2. Testez cet affichage lors des appels à vos différents servlets en passant des paramètres quelconques
3. Ecrire une servlet S1 qui est immédiatement redirigée vers une servlet S2
 - (a) Vérifiez ce qui se passe lors d'une redirection via `sendRedirect`
 - (b) Vérifiez ce qui se passe lors d'une redirection via un `RequestDispatcher`

Exercice 6 : MySessionCounterListener

Q1. Réalisez une classe `MySessionCounterListener` qui affiche dans la console, le nombre de sessions actives dans le contexte. On implémentera pour cela un `HttpSessionListener` avec un attribut `cpt` global.

Pour tester,

1. Réutilisez votre compteur de l'exo1 ou créez une servlet (ou JSP) toute simple qui initialise une session. Pour faciliter le test on mettra la durée de vie de la session à 10sec.
2. Se connecter via plusieurs navigateurs et/ou machines à cette page afin d'augmenter le nombre de sessions.

Les messages doivent s'incrémenter au fur et à mesure des nouveaux accès. Si vous ne touchez plus à rien, les sessions disparaissent toutes seules peu à peu.

Exercice 7 : MyAttributeListener

Q1. Réalisez une classe `MyAttributeListener` qui indique via un message dans la console quand un attribut a été rangé, modifié ou supprimé dans la session. Affichez dans le même message de quel attribut il s'agit et avec quelle valeur. On utilisera pour cela un `HttpSessionAttributeListener`

Exercice 8 : MyContextListener

Q1. Réalisez une classe `MyContextListener` qui crée une connexion au SGBD lors de l'initialisation du contexte et ferme cette connexion lors de la destruction du contexte. Cette connexion sera rangée dans le contexte d'application. On utilisera pour cela un `ServletContextListener`.

Afin de tester ce listener, créez une servlet toute simple qui récupère la connexion et l'utilise pour se connecter à la base de données.